

Data-Oriented Programming (DOP) Quick Reference

4 Core Principles

1. **Model Data Only** → Use records & sealed types, no behavior in data
 2. **Make Data Immutable** → Records are immutable by default
 3. **Validate at Boundary** → Validate in constructors, throw exceptions
 4. **Make Illegal States Unrepresentable** → Use sealed types & enums
-

When to Use Each Pattern

Pattern 1: Transaction/Purchase System (Concert Tickets)

Use when you see:

- Purchases, bookings, reservations
- Advisory/notification rules
- Multi-criteria search
- Geographic/location data
- Computed properties based on rules

Key Features:

- Advisories computed from rules
 - Search with Predicates
 - Country/location metadata
 - Validation: prices, dates, capacities
-

Pattern 2: Application/Workflow System (Security Clearance)

Use when you see:

- Applications, approvals, requests
- Status transitions (PENDING → APPROVED/REJECTED)
- Hierarchical levels (Level 1, 2, 3)

- Reviewer/approval process
- State validation rules

Key Features:

- Status workflow enforcement
 - Result<T> for success/failure
 - Hierarchical comparisons
 - Illegal state prevention
-

Code Structure Template

Step 1: Define Enums

```
java
enum YourChoiceType {
    OPTION_1, OPTION_2, OPTION_3;
    // Add value() and description() if hierarchical
}
```

Step 2: Create Records (Entities)

```
java
record YourEntity(
    String id,
    String name,
    OtherType field
){
    // Constructor with auto-ID
    public YourEntity(String name, OtherType field) {
        this(UUID.randomUUID().toString(), name, field);
    }

    // Validation in compact constructor
    public YourEntity {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name required");
        }
    }
}
```

Step 3: Sealed Interface for Operations (BONUS!)

```

java
sealed interface YourOperation<T> permits
    YourOperation.Action1,
    YourOperation.Action2 {

    record Action1(/* params */) implements YourOperation<ReturnType1> {}
    record Action2(/* params */) implements YourOperation<ReturnType2> {}
}

```

Step 4: Static Methods (System Class)

```

java
class YourSystem {
    public static ReturnType performAction(YourOperation.Action1 op) {
        // Process data
        // Return result
    }
}

```

Design Patterns Quick Reference

Factory Pattern

```

java
public static Ticket createTicket(Concert concert, Attendee attendee, double price) {
    return new Ticket(concert, attendee, price);
}

```

Singleton Pattern

```

java
class Config {
    private static final Config INSTANCE = new Config();
    private Config() {}
    public static Config getInstance() { return INSTANCE; }
}

```

Result Type (for error handling)

```

java
sealed interface Result<T> permits Result.Success, Result.Failure {

```

```
record Success<T>(T value) implements Result<T> {}
record Failure<T>(String error) implements Result<T> {}
}
```

Validation Checklist

Always Validate:

- Non-null/non-blank strings
- Non-negative numbers (prices, capacities)
- Future dates (if required)
- Minimum/maximum values
- Status transitions
- Hierarchical comparisons

Validation Location:

```
java
record Entity(String field) {
    public Entity { // ← Compact constructor
        if (field == null) {
            throw new IllegalArgumentException("Field required");
        }
    }
}
```

Updating Immutable Records

```
java
// To "update" a record, create a new instance
var updatedList = new ArrayList<>(original.list());
updatedList.add(newItem);

var updated = new YourRecord(
    original.id(),
    original.name(),
    updatedList // New list
);
```

Pattern Matching Examples

Switch Expressions (Java 17+)

```
java
return switch (result) {
    case Result.Success<T> s -> s.value();
    case Result.Failure<T> f -> throw new RuntimeException(f.error());
};
```

Instanceof Pattern

```
java
if (result instanceof Result.Success<Application> success) {
    Application app = success.value();
    // Use app
}
```

Common Exam Scenarios

Scenario: Computed Properties (Advisories, Warnings)

1. Create sealed interface for all possible advisories
2. Compute in record constructor or static method
3. Store as immutable list in record

Scenario: Status Workflow

1. Use enum for status
2. Validate transitions in operation methods
3. Return Result<T> for operations that can fail

Scenario: Search/Filter

1. Accept Predicate<T> in operation
2. Use streams with filter
3. Return immutable list

Scenario: Hierarchical Levels

1. Add numeric value to enum

2. Add comparison methods (isHigherThan)
 3. Validate in operations
-

Quick Tips for Exam

1. **Always use sealed interface with generics** → Bonus points!
 2. **Validate everything in constructors** → Show DOP principle 3
 3. **Use pattern matching** → Shows modern Java knowledge
 4. **Add helper methods to enums** → value(), description(), comparisons
 5. **Make lists immutable** → `Collections.unmodifiableList()`
 6. **Use UUID for auto-generated IDs** → `UUID.randomUUID().toString()`
 7. **Include design patterns** → Factory, Singleton, Result type
 8. **Comment your code** → Explain DOP principles used
-

Exam Day Strategy

1. **Read spec carefully** → Identify pattern (Transaction vs Workflow)
 2. **List all enums** → Fixed choices become enums
 3. **Draw entity relationships** → Helps structure records
 4. **Identify rules** → Become validation logic
 5. **List operations** → Become sealed interface methods
 6. **Write main() test** → Shows working system
-

Common Mistakes to Avoid

-  Forgetting to make lists immutable
 -  Not validating in constructors
 -  Using mutable fields in records
 -  Allowing invalid state transitions
 -  Forgetting generic types in sealed interfaces
 -  Not handling null/empty values
 -  Missing UUID import: `import java.util.UUID;`
-

Imports You'll Need

```
java
import java.time.LocalDateTime;
import java.time.Month;
import java.util.*;
import java.util.function.Predicate;
import java.util.stream.*;
```



Remember

"Code doesn't need to compile perfectly - focus on DOP design and patterns!"

- Dr. John Healy

Show you understand:

- ☒ Sealed interfaces with generics
- ☒ Records for immutable data
- ☒ Validation at boundaries
- ☒ Pattern matching
- ☒ Design patterns
- ☒ Stateless processing